



自然语言处理 · 实验一

文本分类实验报告

姓名：禹相祐

学号：2312900

学院：计算机学院

实验目标与统一设置

本实验围绕 NYT 新闻的 business、politics、sports 三分类，依次比较词袋表示、平均词向量表示和 BERT 微调。前两类表示采用 Logistic Regression，BERT 联合训练编码器与分类层，统一使用测试集 Accuracy 和 Macro-F1 评价。

NYT 共 **11,519** 篇新闻，以种子 **42** 随机打乱后按 80% / 10% / 10% 划分为 **9,215 / 1,152 / 1,152** 篇，三个任务复用同一划分。测试集三个类别依次为 **148 / 165 / 839** 篇。Accuracy 衡量整体预测正确率；Macro-F1 对各类别的 F1 等权平均，有助于观察样本较少类别的效果。

Task 1 • Bag of Words

1. 任务与原理

仅用训练集构建大小为 **61,329** 的词表，将文本转换为稀疏向量。Binary Bag of Words 用 0/1 表示词是否出现；Word Frequency 保存各词在文档中的原始出现次数。两种表示均忽略词序，由多项逻辑回归通过 softmax 输出类别概率，并最小化带 L2 正则项的交叉熵损失。

预处理采用小写转换和正则分词，保留单字符词及数字、忽略标点；测试集中未出现在训练词表中的词不计入特征。分类器参数为 $C=1.0$ 、`solver="lbfgs"`、`max_iter=2000`、`tol=1e-4`。

2. 关键代码

以下代码实现稀疏向量构建与分类；binary 分别取 True、False 运行两组实验。

```
vocabulary = build_vocabulary(texts[splits["train"]]) # 仅用训练集建词表
def vectorize(texts, vocabulary, binary=False):
    indices, values, indptr = [], [], [0]
    for text in texts:
        counts = Counter(tokenize(text))
        entries = sorted((vocabulary[w], c) for w, c in counts.items()
                        if w in vocabulary) # 忽略词表外的词
        for column, count in entries:
            indices.append(column)
            values.append(1 if binary else count) # 二值或原始词频
        indptr.append(len(indices))
    return csr_matrix((values, indices, indptr),
                      shape=(len(texts), len(vocabulary)), dtype=np.float64)
matrices = {part: vectorize(texts[idx], vocabulary, binary)
            for part, idx in splits.items()} # 各集合共享同一词表
model = LogisticRegression(**parameters)
model.fit(matrices["train"], labels[splits["train"]])
predictions = model.predict(matrices["test"])
result = evaluate(labels[splits["test"]], predictions, classes)
print(result["accuracy"], result["macro_f1"])
```

3. 实验结果与分析

文本表示	验证集 Accuracy	测试集 Accuracy	测试集 Macro-F1
Binary Bag of Words	98.5243%	98.6979%	0.971379
Word Frequency	98.5243%	98.8715%	0.975509

词频表示正确分类 **1,139 / 1,152** 篇，比二值表示净多 **2** 篇。逐条比较发现，两者有 **16** 篇预测不同：词频表示纠正了二值表示的 **9** 个错误，同时将原本判对的 **7** 篇判错；另有 **6** 篇是两者共同判错的。因此，这次结果只能说词频表示略优，尚不能据此认定词语重复次数带来了稳定收益。

16 篇分歧中，business、politics、sports 分别占 **8、6、2** 篇。差异主要出现在前两类，后续更值得检查这些新闻是否同时涉及商业与政治主题，以及重复词是否干扰了判断。这里尚未核读新闻内容，无法确定具体误判原因。两种方法分别迭代 **19、132** 次收敛；词频模型的微小指标提升伴随着更多优化迭代，但本实验没有测量两者耗时。

Task 2 • Word Embedding / Word2Vec

1. 任务与原理

本任务比较三组 **100 维**词向量：预训练 glove.6B.100d、在 **90,000** 篇 AG News 文本上训练的 Word2Vec，以及仅在 **9,215** 篇 NYT 训练文本上训练的 Word2Vec。GloVe 根据全局词共现统计学习表示；本实验的 Word2Vec 使用 Skip-gram，通过预测中心词的上下文并结合负采样学习表示。

文档向量为有效词向量的平均： $\mathbf{e}(\mathbf{d}) = \sum \mathbf{e}(\mathbf{w}_i) / n$ 。重复词按出现次数参与平均，未知词跳过，无有效词时保留零向量。Word2Vec 统一设置 window=5、min_count=2、negative=5、epochs=10、workers=1、seed=42。三组文档向量均输入与 Task 1 参数相同的逻辑回归；NYT 验证集和测试集不参与 Word2Vec 训练。

2. 关键代码

以下代码实现词向量准备、文档平均和分类；name 依次遍历三组方法。

```
if name == "GloVe":
    needed = {w for doc in documents for w in doc}
    vectors, _ = load_glove(args.glove, needed) # 仅查表，不更新预训练向量
elif name == "Word2Vec_AG":
    vectors, _ = train_word2vec(read_ag(ROOT / "data/ag.csv"))
else:
    corpus = [documents[i] for i in splits["train"]] # 仅用训练文本
    vectors, _ = train_word2vec(corpus)
def mean_features(docs, vectors):
    features = np.zeros((len(docs), 100), dtype=np.float64)
    for i, words in enumerate(docs):
        valid = [vectors[w] for w in words if w in vectors] # 重复词保留，未知词跳过
        if valid:
            features[i] = np.mean(valid, axis=0, dtype=np.float64)
    return features # 无有效词的文档保留零向量
matrices = {p: mean_features([documents[i] for i in idx], vectors)
             for p, idx in splits.items()}
classifier = LogisticRegression(**LR_PARAMETERS)
classifier.fit(matrices["train"], labels[splits["train"]])
predictions = classifier.predict(matrices["test"])
```

3. 实验结果与分析

词向量来源	测试词元覆盖率	测试集 Accuracy	测试集 Macro-F1
GloVe 6B	99.7860%	98.6111%	0.970928
Word2Vec · AG News	97.3778%	97.3090%	0.944442
Word2Vec · NYT 训练集	99.2908%	98.0035%	0.959598

AG News 有 **90,000** 篇，NYT 训练集只有 **9,215** 篇，但按相同方式分词后，两者分别有 **3,581,128**、**6,148,820** 个词元。按篇数看 AG 更大，按实际输入词元数看 NYT 更多，因此不能把这组结果解释成“小语料胜过大语料”。NYT 模型还同时具有更高的测试词元覆盖率和更接近目标任务的语料分布，这些因素在当前比较中混在一起。

NYT Word2Vec 比 AG 模型多判对 **8** 篇，GloVe 又比 NYT 模型多判对 **7** 篇。覆盖率与分类效果的排序一致，但尚未在共同词表上比较，无法分开判断覆盖率和已知词向量质量的贡献。GloVe 与自训练模型的训练语料、语料规模和学习目标也不同；本次可以确定它在这三组配置中效果最好，无法将优势单独归因于 GloVe 算法。

Task 3 • BERT Fine-tuning

1. 任务与原理

使用 google-bert/bert-base-uncased 进行三分类微调，以双向自注意力编码上下文，通过交叉熵联合更新编码器和分类层。**max_length=64**（含特殊标记）和 **3 epochs** 均按作业要求设置。测试使用完成第三轮后的模型，没有按验证指标回退选取检查点，也未比较其他长度和训练轮数。

训练/评价批量为 **16/32**，AdamW 学习率 2×10^{-5} 、权重衰减 **0.01**，线性调度、无 warmup，梯度裁剪阈值 **1.0**。这些是本次固定配置，未做参数对照。实验使用 PyTorch **2.8.0**、Transformers **4.55.4** 和 MPS，共 **1,728** 个训练步。

2. 关键代码

train_loader 由分词结果和标签构建；优化器与调度器按上述配置初始化。

```
encoded = tokenizer(selected_texts, truncation=True, padding="max_length",
                    max_length=64, return_tensors="pt") # 超长文本截断，含特殊标记
for epoch in range(1, 4): # 按要求完整训练三轮
    model.train()
    for input_ids, attention_mask, labels in train_loader:
        optimizer.zero_grad(set_to_none=True)
        output = model(input_ids=input_ids.to(device),
                       attention_mask=attention_mask.to(device),
                       labels=labels.to(device))
        output.loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
    validation, _ = evaluate_model(model, validation_loader, device, classes)
    test, predicted_ids = evaluate_model(model, test_loader, device, classes)
# 测试使用第三轮结束时的参数，验证集仅用于监测
```

3. 实验结果与综合比较

阶段	训练损失	Accuracy	Macro-F1
第 1 轮 · 验证集	0.143156	97.1354%	0.927347
第 2 轮 · 验证集	0.045595	96.7014%	0.926559
第 3 轮 · 验证集	0.015009	97.3958%	0.935605
第 3 轮结束 · 测试集	—	97.5694%	0.946222

BERT 共判错 **28** 篇，词频模型判错 **13** 篇。按真实类别统计，business 的错误数为 **10 对 9**，politics 为 **16 对 4**，sports 为 **2 对 0**；净差距主要集中在 politics。训练损失下降时，验证损失由 **0.107484** 升至 **0.127799**，但第三轮验证 Accuracy 和 Macro-F1 仍最高。这说明分类判定与概率损失的变化并不一致，单凭损失上升还不能断言继续训练必然变差。

词袋不利用词序，仍达到 **98.87%** 准确率，提示主题词可能已能区分大部分新闻。不过词袋读取全文，BERT 只保留前 **64** 个 token，两者获得的信息量不同。截断是否导致 politics 类多错，目前无法判断；应优先比较被截断和未被截断新闻的错误率，再做更长输入的对照。此次六组结果支持词频模型在当前设置下领先，尚不足以判断完整输入下 BERT 与词袋的优劣。